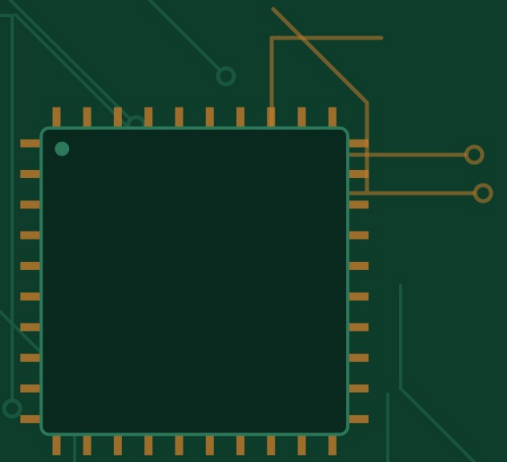
PERTEMUAN 02

Dasar Pemrograman C/C++ untuk Mikrokontroler

Tata bahasa untuk "berbicara" dengan ESP32



Sistem Tertanam (Embedded Systems) · S1 Teknik Elektro

Dosen Pengampu: Anton Prafanto

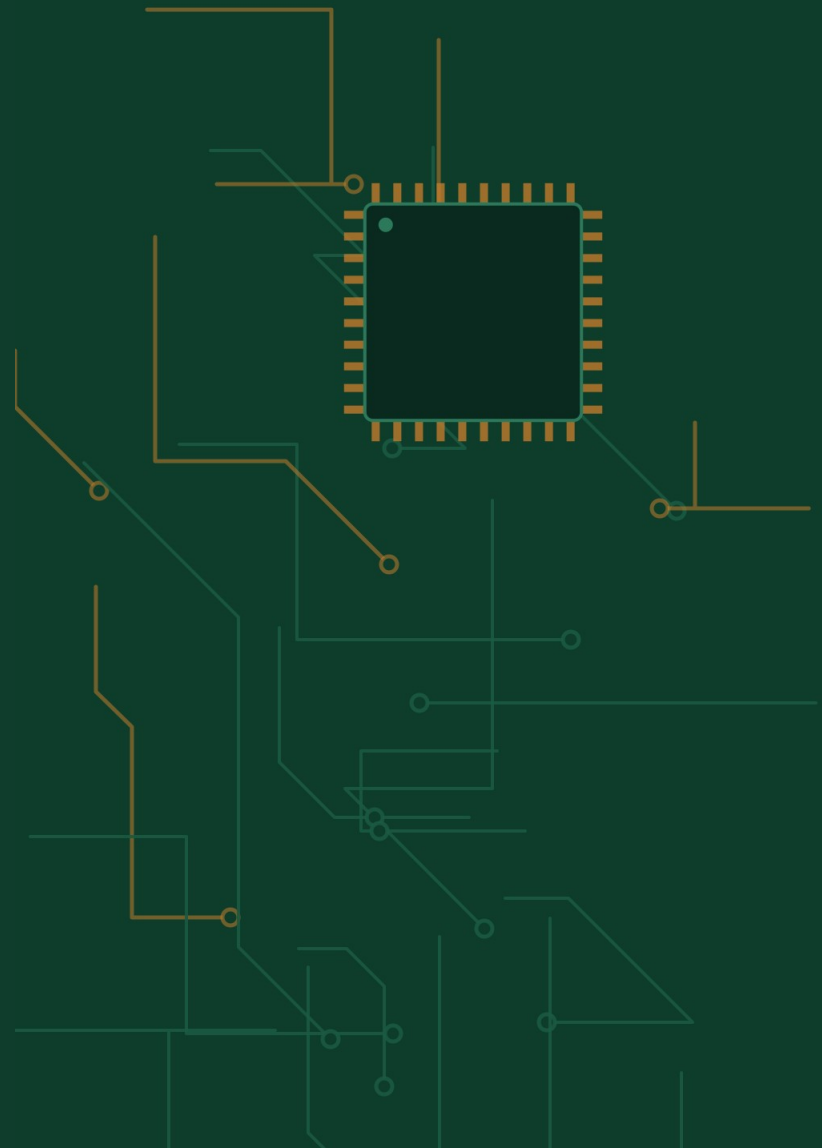
Tujuan Pembelajaran

- Membaca struktur program Arduino: `setup()`, `loop()`, **prototipe**
- Memilih **tipe data** yang tepat dan menghindari jebakan pemula
- Memakai operator aritmetika, logika, dan **operator bit**
- Menyusun **percabangan**, **perulangan**, **fungsi**, dan **larik**
- Mencetak dan membaca data lewat **Serial Monitor**

01

C/C++ dan Struktur Program

Dari kode yang kamu tulis sampai instruksi yang dijalankan CPU



Dari Kode C++ ke Bahasa Mesin

1 Kode yang kamu tulis (C++)

```
float nilaiMaksimum(const float data[],
                    int jumlah) {
  float terbesar = data[0];
  for (int i = 1; i < jumlah; i++) {
    if (data[i] > terbesar) {
      terbesar = data[i];
    }
  }
  return terbesar;
}
```

Mudah dibaca manusia: ada nama, kata kunci, dan struktur yang jelas.

Build

Kompilator

GCC untuk Xtensa, dijalankan PlatformIO saat kamu klik **Build**

Pustaka Arduino-ESP32

`digitalWrite()`, `Serial`, `delay()`, … ikut diterjemahkan

hasil

2 Hasil terjemahan (bahasa mesin)

```
004136  entry   a1, 32
000203  lsi     f0, a2, 0
180c    movi.n  a8, 1
042837  blt     a8, a3, ...
a09820  addx4   a9, a8, a2
000913  lsi     f1, a9, 0
4b0010  olt.s   b0, f0, f1
db0100  movt.s  f0, f1, b0
  …       (dan seterusnya)
```

diunggah ke flash

CPU Xtensa ESP32 mengambil dan menjalankan instruksi itu satu per satu, ratusan juta kali per detik.

Ikon: Font Awesome Free (CC BY 4.0)

main() di Laptop vs setup()/loop() di ESP32



Program C di Laptop

- Dimulai dari `main()`
- Berjalan dari atas ke bawah
- `return 0;` lalu **selesai**
- Sistem operasi menunggu program selesai



Program Arduino di ESP32

- `app_main()` membuat *task* `loopTask`
- `setup()` dipanggil **sekali**
- `loop()` diulang **selamanya**
- Tidak pernah selesai selama ada listrik

Anatomi Program Arduino

```
1  // Program: LED berkedip sambil menghitung
2  /* Komentar blok boleh beberapa baris.
3     Semua komentar diabaikan kompilator. */
4  #include <Arduino.h>
5
6  const int PIN_LED = 2;
7  int hitungan = 0;
8
9  void setup() {
10   Serial.begin(115200);
11   pinMode(PIN_LED, OUTPUT);
12 }
13
14 void loop() {
15   hitungan++;
16   digitalWrite(PIN_LED, HIGH);
17   delay(500);
18   digitalWrite(PIN_LED, LOW);
19   delay(500);
20 }
```

- **Komentar satu baris** diawali `//`
- **Komentar blok** di antara `/*` dan `*/`
- **#include**: memanggil pustaka Arduino
- **Konstanta global**: nilainya tidak bisa diubah
- **Variabel global**: dikenal di semua fungsi
- **Fungsi setup()**: dijalankan SEKALI; `{` membuka blok
- `}` menutup blok; isi di antara `{ }` = satu **blok**
- **Fungsi loop()**: diulang TERUS selama board menyala
- **Pernyataan** selalu diakhiri titik koma `;`
- Memanggil fungsi: **nama(argumen1, argumen2)**

Peka huruf besar-kecil! `digitalWrite` ≠ `digitalwrite`, `HIGH` ≠ `High`, `hitungan` ≠ `Hitungan`. Spasi dan baris kosong bebas, tetapi indentasi (teks menjorok) membuat kode mudah dibaca.

Prototipe Fungsi di main.cpp

src/main.cpp

```cpp
#include <Arduino.h>

float luasLingkaran(float r);   // prototipe

void setup() {
  Serial.begin(115200);
  Serial.println(luasLingkaran(2.0));
}

void loop() {
}

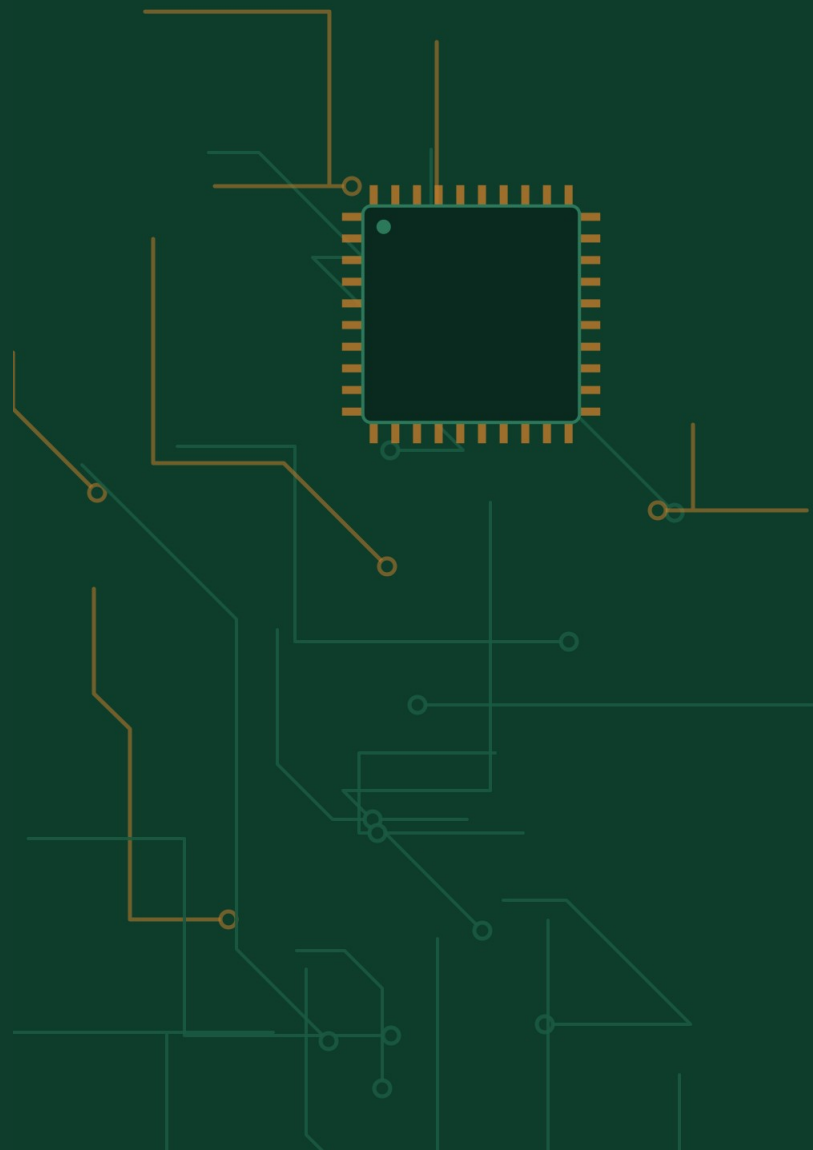
float luasLingkaran(float r) {  // definisi
  return 3.14159 * r * r;
}
```

- `sketch.ino` (Wokwi): include dan prototipe dibuat **otomatis**
- `src/main.cpp` (PlatformIO): **wajib** ditulis sendiri
- Tanpa prototipe: galat *was not declared in this scope*
- Gaya PlatformIO tetap sah disalin ke Wokwi

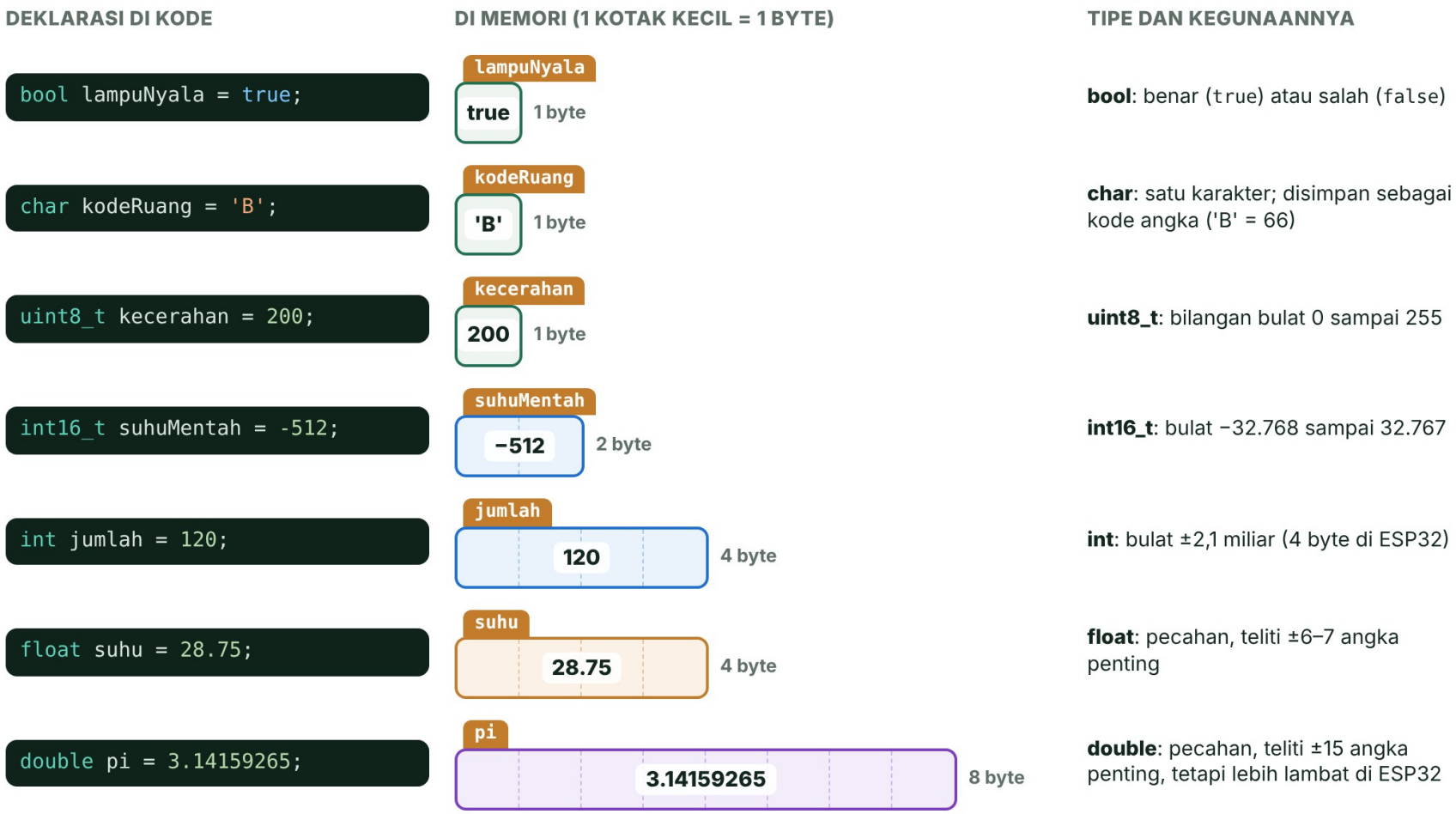
02

Data dan Operator

Variabel, tipe data, jebakan pemula, dan operator bit



Variabel = Kotak Berlabel di Memori

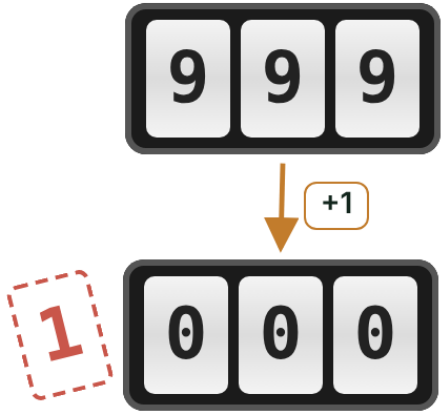


Ukuran Tipe Data di ESP32

Tipe	Ukuran	Rentang / isi
bool, char, uint8_t	1 byte	true/false · 1 karakter · 0–255
int16_t	2 byte	−32.768 sampai 32.767
int, long, int32_t	4 byte	±2,1 miliar
uint32_t	4 byte	0 sampai 4.294.967.295
float	4 byte	Pecahan, ±6–7 angka penting
long long, double	8 byte	$\pm 9{,}2 \times 10^{18}$ · pecahan ±15 angka penting

Jebakan 1: Overflow

Odometer 3 digit



Angka 1 di depan **tidak punya tempat** (hanya ada 3 digit), jadi yang tampak kembali ke 000.

Di dalam ESP32: `uint8_t` 255 + 1

```
  1111 1111   ← 255 (8 bit penuh)
+ 0000 0001   ← 1
──────────────
1 0000 0000   ← 256 butuh 9 bit
  0000 0000   ← yang tersimpan: 0
```

Bit ke-9 (**merah**) dibuang karena kotak `uint8_t` hanya 8 bit. Sebaliknya, 0 − 1 menjadi 255.

```
uint8_t angka = 255;
angka = angka + 1;   // jadi 0, bukan 256!
```

Cara menghindarinya: pilih tipe yang cukup besar untuk nilai terbesar yang mungkin muncul. Untuk tipe bertanda (misalnya `int`), meluap justru **tidak terdefinisi** dalam C/C++, jadi jangan diandalkan.

Empat Jebakan Lain



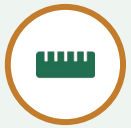
Pembagian bulat

`7 / 2` = 3. Tulis `7 / 2.0` agar hasilnya 3,5.



Modulo %

Sisa bagi: `7 % 2` = 1. Untuk genap/ganjil dan indeks berputar.



Konversi tipe

`int x = 3.99;` menjadi 3 (dipotong). Pakai `round()`.



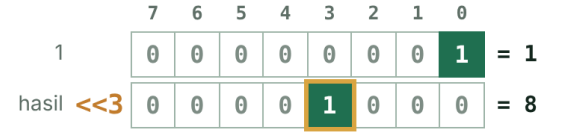
Pecahan dan ==

36,6 tersimpan 36,5999…; bandingkan dengan toleransi.

Operator Bit: Sakelar Satu per Satu

1. Geser ke kiri <<

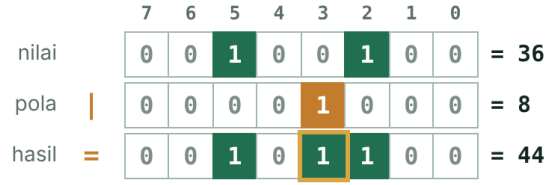
```
1 << 3
```



Angka 1 bergeser 3 tempat ke kiri: 1 menjadi 8.

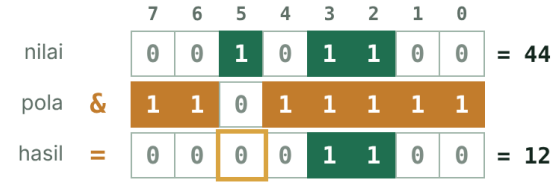
2. Menyalakan bit 3 dengan | (OR)

```
nilai = nilai | (1 << 3);
```



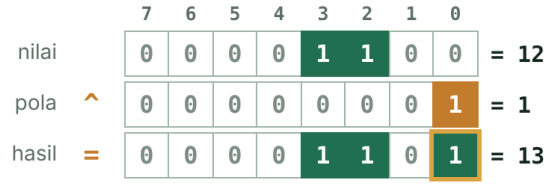
3. Mematikan bit 5 dengan & ~ (AND NOT)

```
nilai = nilai & ~(1 << 5);
```



4. Membalik bit 0 dengan ^ (XOR)

```
nilai = nilai ^ (1 << 0);
```



Aturan per bit

A	B	A & B	A | B	A ^ B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Kenapa penting di sistem tertanam?

Di dalam ESP32, banyak pengaturan disimpan sebagai bit di **register**. Satu bit bisa berarti satu pin GPIO: 1 = HIGH, 0 = LOW. Pola `1 << n` memilih bit ke-n, misalnya bit milik satu pin.

~ membalik semua bit (NOT). Jangan tertukar: & dan | bekerja per bit, sedangkan && dan || bekerja pada benar/salah.

Empat Resep Operator Bit

kode/02-operator-dan-bit

```
uint8_t lampu = 0b00000000;

lampu |=  (1 << 3);   // nyalakan bit 3
lampu &= ~(1 << 3);   // padamkan bit 3
lampu ^=  (1 << 1);   // balik bit 1

int aktif = (lampu >> 7) & 1;  // baca bit 7

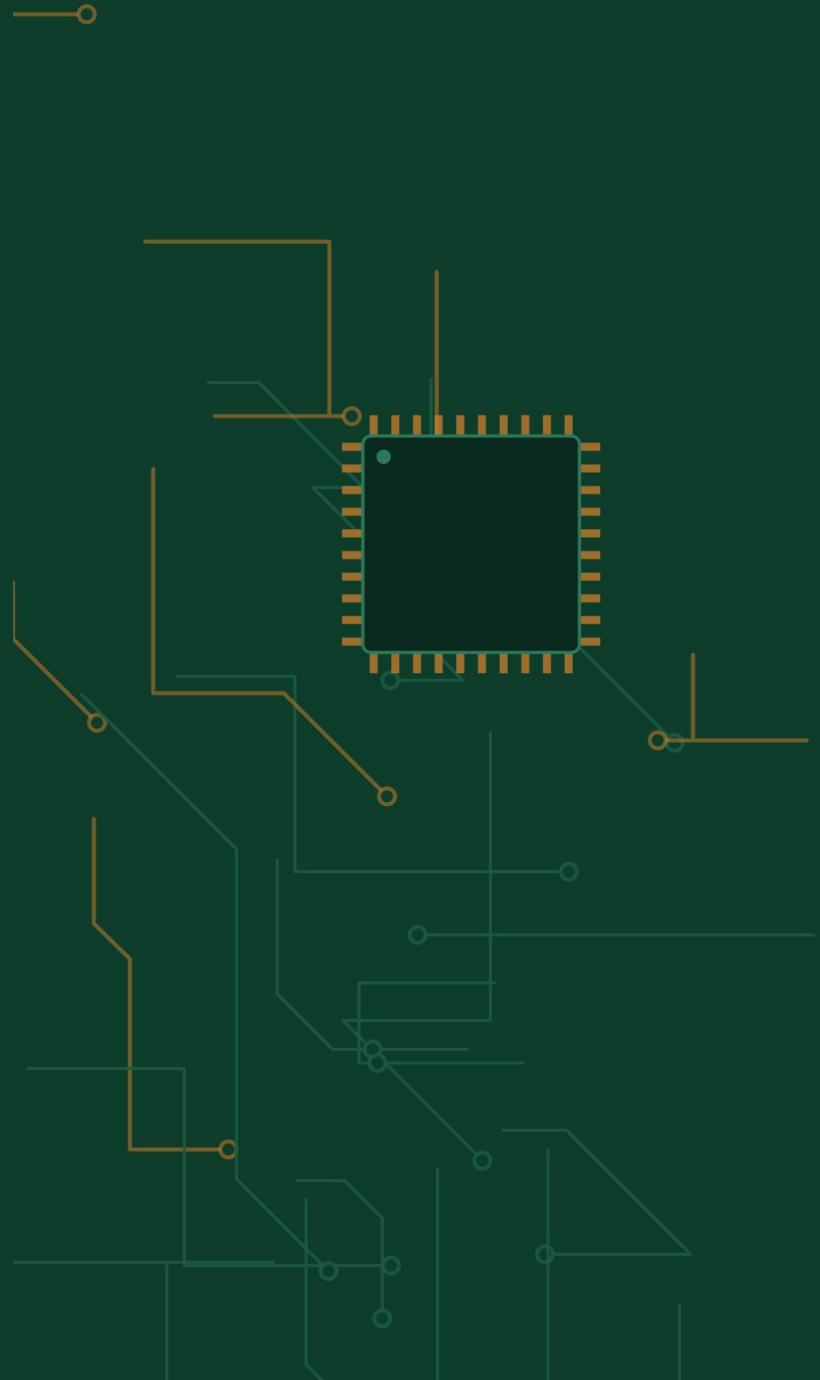
// & | bekerja per bit
// && || bekerja pada benar/salah
```

- |= menyalakan, bit lain tetap
- &= ~ memadamkan satu bit saja
- ^= membalik (*toggle*)
- = mengisi, == membandingkan

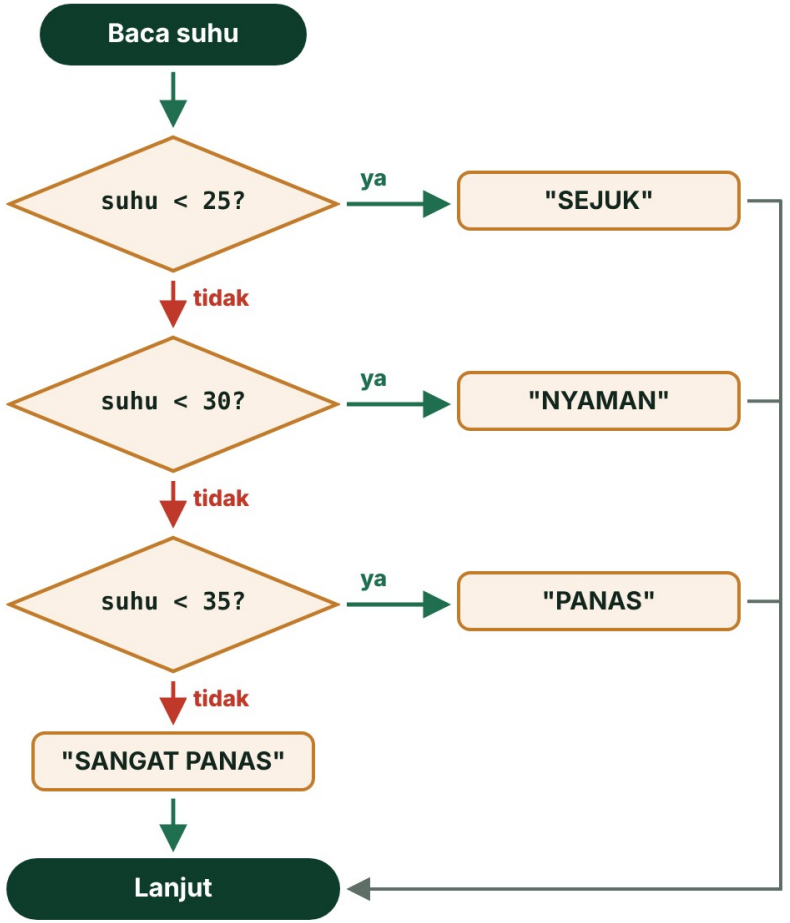
03

Alur Program, Fungsi, dan Serial

Mengambil keputusan, mengulang, dan berbicara lewat Serial Monitor



Percabangan if / else if / else




```
if (suhu < 25.0) {
  Serial.println("SEJUK");
} else if (suhu < 30.0) {
  Serial.println("NYAMAN");
} else if (suhu < 35.0) {
  Serial.println("PANAS");
} else {
  // semua syarat di atas salah
  Serial.println("SANGAT PANAS");
}
```

Contoh: suhu = 27,0

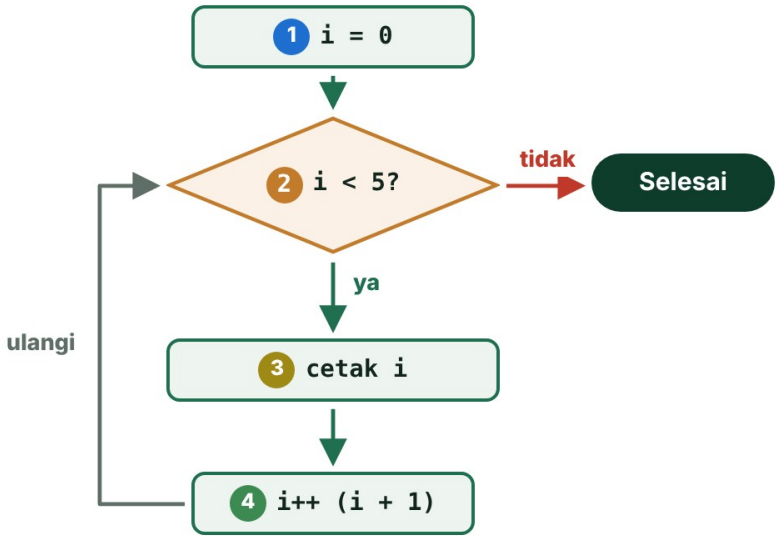
`27 < 25`? **tidak** → `27 < 30`? **ya** → cetak "NYAMAN". Syarat `< 35` tidak diperiksa lagi.

Bentuk diagram alir: oval = mulai/selesai, belah ketupat = keputusan (ya/tidak), persegi = tindakan.

Perulangan for

```
for (int i = 0; i < 5; i++) {  Serial.println(i);  }
```

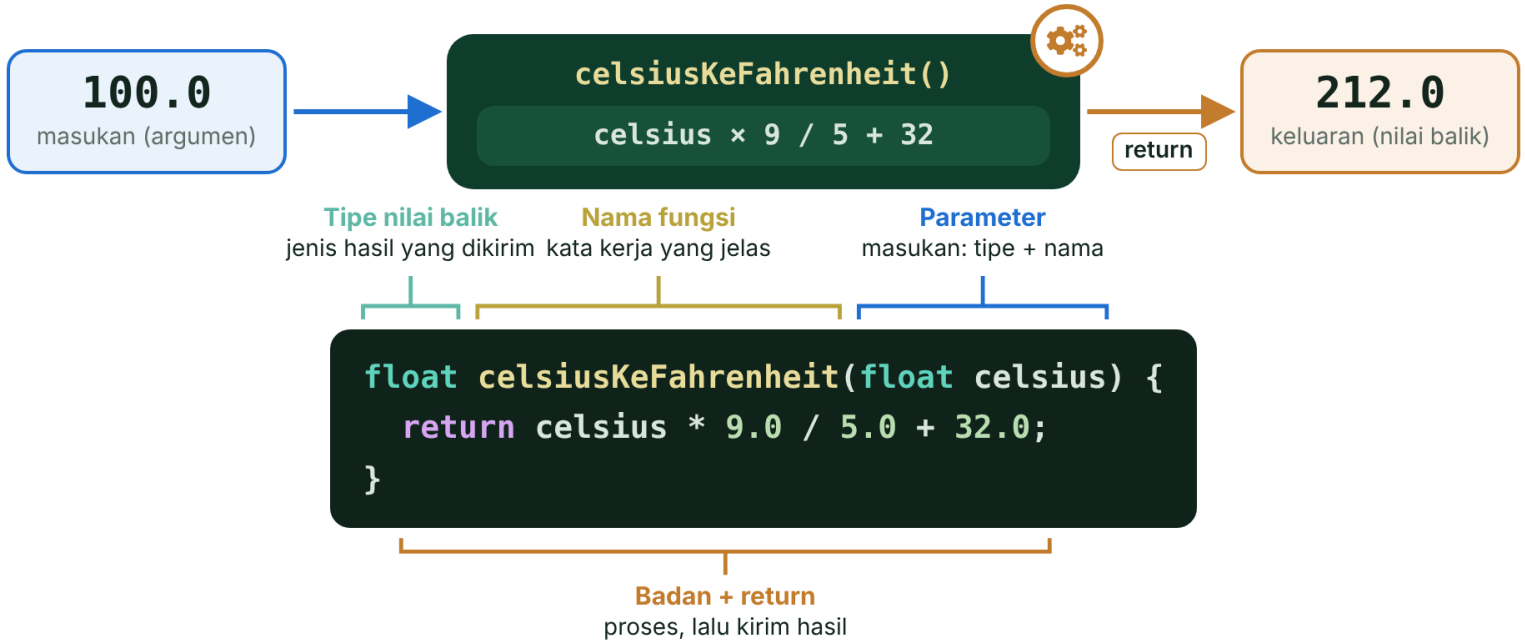
1. **Inisialisasi** — sekali, di awal
2. **Syarat** — dicek sebelum tiap putaran
3. **Badan** — bagian yang diulang
4. **Pembaruan** — setelah tiap putaran



Putaran	Nilai i	i < 5?	Tercetak
1	0	ya	0
2	1	ya	1
3	2	ya	2
4	3	ya	3
5	4	ya	4
–	5	**tidak**	berhenti

Badan perulangan berjalan **5 kali** (i = 0 sampai 4). Pola `i = 0; i < N` selalu menghasilkan tepat **N** putaran, cocok dengan indeks larik yang juga mulai dari 0.

Fungsi = Mesin Kecil



Memanggil fungsi

`float f = celsiusKeFahrenheit(100.0);`
Nilai 100,0 masuk ke parameter `celsius`; hasil 212,0 disimpan di `f`.

Prototipe (deklarasi)

`float celsiusKeFahrenheit(float celsius);`
Ditulis di atas berkas agar fungsi boleh dipanggil sebelum isinya ditulis.

Ikon: Font Awesome Free (CC BY 4.0)

Larik: Indeks Mulai dari 0

```
float pembacaanSuhu[] = {26.5, 27.8, 29.4, 31.6, 30.2, 28.0};
```



Membaca satu isi

`pembacaanSuhu[3]` bernilai **31,6**: isi ke-4, karena hitungan dimulai dari 0.

Indeks [6] tidak ada!

6 isi berarti indeks 0 sampai 5. Membaca `[6]` mengambil memori milik data lain: hasilnya acak atau program *crash*.

Menghitung jumlah isi

`sizeof(pembacaanSuhu) / sizeof(pembacaanSuhu[0])`
= 24 byte / 4 byte = **6**

Kode Format Serial.printf()

Kode	Untuk	Contoh → hasil
%d	int	printf("%d", -12) → -12
%u	bulat tanpa tanda, sizeof	sizeof(float) → 4
%ld / %lu	long / unsigned long, uint32_t	4294967295UL → 4294967295
%f / %.2f	float, double	28.756 → 28.756000 / 28.76
%s	teks char[], String + .c_str()	"OK" → OK
%c	satu karakter	'B' → B
%x / %X	heksadesimal	255 → ff / FF

Membaca Input dari Serial Monitor



Serial Monitor PlatformIO

Setiap huruf langsung dikirim begitu diketik. Tambahkan di `platformio.ini`:
`monitor_echo = yes` → ketikanmu tampak di layar.
`monitor_eol = LF` → Enter dikirim sebagai **\n** (bawaannya CR+LF).

Serial Monitor Wokwi

Ketik satu baris di Serial Monitor, lalu tekan Enter. Wokwi mengirim baris itu dan menambahkan **\n** secara bawaan.

Batas waktu

`readStringUntil()` juga berhenti bila tidak ada huruf baru selama batas waktu (bawaan 1 detik). Praktikum 5 memperpanjangnya dengan `Serial.setTimeout(20000)`.

Kuis Cepat



`uint8_t x = 250;` lalu `x = x + 10;`. Berapa nilai `x`?

A 260

B 4

C 255

D 10

Rangkuman

- Di ESP32, *task* `loopTask` menjalankan `setup()` sekali, lalu `loop()` selamanya
- Di `main.cpp`: wajib `#include <Arduino.h>` dan prototipe fungsi
- Di ESP32, `int` dan `float` berukuran 4 byte, `double` 8 byte; cek dengan `sizeof()`
- Waspadai *overflow*, `7 / 2` = 3, pemotongan, dan `==` pada pecahan
- Operator bit: `|=` menyalakan, `&= ~` memadamkan, `^=` membalik bit
- Cetak dengan `printf()` dan kode format; baca input dengan `readStringUntil()`, `trim()`, lalu `toInt()`

Sebelum Pertemuan 3

Tugas

- Kerjakan Latihan 1–3 di modul
- Coba tantangan tebak angka

Persiapan

- Baca modul Pertemuan 3: GPIO Output
- Bawa *board*, LED, dan resistor bila punya

Butuh Bantuan?

- Baca panduan/06-mengatasi-masalah.md
- Tanyakan di forum kelas